

Trabajos en Clase

Programación en C — 30 programas

Índice de Programas

01. Introducción a Arreglos — Recorrido y Visualización
02. Arreglos — Contar Números Pares
03. Arreglos — Cálculo de Promedio
04. Arreglos — Carga con Estructura Repetitiva e Identificación de Pares
05. Comparación de dos Arreglos por Sumatoria
06. Arreglos — Promedio, Mitad del Promedio, Búsqueda por Posición y por Valor
07. Arreglos — Carga, Identificación de Pares y Búsqueda Secuencial
08. Arreglos — Números Aleatorios con Cuadrado y Cubo
09. Arreglos — Números Primos, Recorrido Inverso y Conteo de Ocurrencias (con Funciones)
10. Funciones con Arreglos — Mostrar Elementos y Contar Pares
11. Procedimientos con Arreglos — Mostrar Elementos
12. Cadenas de Caracteres — Declaración y Recorrido
13. Cadenas de Caracteres — Carga e Impresión
14. Búsqueda Binaria en Arreglo Ordenado con Eliminación del Elemento
15. Búsqueda Binaria — Práctica sobre Arreglo Ordenado Ascendente
16. Búsqueda Binaria con Función — Sumatoria, Promedio y Búsqueda en Arreglo
17. Matrices 3×3 — Carga y Visualización
18. Ordenamiento de Matriz 3×3 con Bubble Sort
19. Ordenamiento y Multiplicación de dos Matrices 3×3
20. Matrices 3×3 — Carga sin Repetidos, Máximo, Mínimo y Diagonal Principal
21. Tabla de Máximo Común Divisor (MCD) con los Divisores de 50
22. Arreglos A y B — Suma en C, Ordenamiento, Unión sin Repetidos y Búsqueda Binaria
23. Arreglos — Unión Ordenada Descendente y Búsqueda Binaria (Versión Corregida)
24. Structs — Sistema Básico de Libros y Autores con Menú
25. Structs — Gestión de Libros y Autores con Funciones Separadas
26. Structs — Sistema Completo de Libros y Autores con CRUD y Menú
27. Structs — Carga de Autores y Libros con Struct Anidado
28. Structs — Registro de Clientes con Ordenamiento Burbuja
29. Structs — Clientes con Búsqueda Binaria y Separación por Tipo
30. Structs — Cuentas Bancarias con Informe de Deudores y Acreedores

01. Introducción a Arreglos — Recorrido y Visualización

Archivo: arreglos.c | Declara un arreglo de 9 enteros y los recorre mostrando cada elemento con su índice.

```
#include <stdio.h>

int main() {
    int lista[9] = {0, 4, 78, 5, 32, 9, 77, 1, 23};

    for (int i = 0; i < 9; i++) {
        printf("Digito %d: %d\n", i, lista[i]);
    }

    return 0;
}
```

02. Arreglos — Contar Números Pares

Archivo: arreglos_pares.c | Recorre el arreglo y cuenta cuántos elementos son pares.

```
#include <stdio.h>

int main() {
    int lista[9] = {0, 4, 78, 5, 32, 9, 77, 1, 23};
    int cont = 0;

    for (int i = 0; i < 9; i++) {
        if (lista[i] % 2 == 0)
            cont++;
    }

    printf("Cantidad de numeros pares: %d\n", cont);
    return 0;
}
```

03. Arreglos — Cálculo de Promedio

Archivo: arreglos_promedio.c | Acumula los valores del arreglo y calcula el promedio.

```
#include <stdio.h>

int main() {
    int lista[9] = {0, 4, 78, 5, 32, 9, 77, 1, 23};
    int acum = 0;

    for (int i = 0; i < 9; i++)
        acum += lista[i];

    float prom = (float)acum / 9;
```

```

printf("El promedio es: %.2f\n", prom);
return 0;
}

```

04. Arreglos — Carga con Estructura Repetitiva e Identificación de Pares

Archivo: cargar_elementos.c | Carga 10 elementos con scanf, los muestra e indica cuáles son pares.

```

#include <stdio.h>

int main() {
    int lista[10];

    for (int i = 0; i < 10; i++) {
        printf("Ingresar elemento %d: ", i);
        scanf("%d", &lista[i]);
    }

    for (int i = 0; i < 10; i++) {
        printf("Elemento %d: %d", i + 1, lista[i]);
        if (lista[i] % 2 == 0)
            printf(" <- Es par");
        printf("\n");
    }

    return 0;
}

```

05. Comparación de dos Arreglos por Sumatoria

Archivo: que_arreglo_es_mayor.c | Suma los elementos de dos arreglos y determina cuál es mayor.

```

#include <stdio.h>

int main() {
    int lista1[9] = {0, 4, 79, 5, 32, 9, 77, 1, 23};
    int lista2[9] = {0, 4, 78, 5, 32, 9, 77, 1, 23};
    int acum1 = 0, acum2 = 0;

    for (int i = 0; i < 9; i++) acum1 += lista1[i];
    for (int i = 0; i < 9; i++) acum2 += lista2[i];

    if (acum1 > acum2) printf("El arreglo mayor es el primero con: %d\n", acum1);
    else if (acum2 > acum1) printf("El arreglo mayor es el segundo con: %d\n", acum2);
    else printf("Los arreglos son iguales.\n");

    return 0;
}

```

06. Arreglos — Promedio, Mitad del Promedio, Búsqueda por Posición y por Valor

Archivo: clase_5_5.c | Calcula promedio, muestra elementos mayores a la mitad del promedio, busca por posición y por valor.

```
#include <stdio.h>

int main() {
    int lista[10] = {0, 3, 4, 78, 5, 32, 9, 77, 1, 23};
    int acum = 0;

    for (int i = 0; i < 10; i++) {
        printf("%d", lista[i]);
        if (i < 9) printf("--");
        acum += lista[i];
    }
    printf("\n");

    float prom = (float)acum / 10;
    float mitadPromedio = prom / 2;

    printf("Promedio: %.2f\n", prom);
    printf("Elementos mayores a la mitad del promedio (%.2f): ", mitadPromedio);
    for (int i = 0; i < 10; i++)
        if (lista[i] > mitadPromedio) printf("%d ", lista[i]);
    printf("\n");

    /* Búsqueda por posicion */
    int posicion;
    printf("Ingrese una posicion (0-9): ");
    scanf("%d", &posicion);
    if (posicion >= 0 && posicion < 10)
        printf("En la posicion %d se encuentra el numero %d\n", posicion,
lista[posicion]);
    else
        printf("Posicion fuera de rango.\n");

    /* Búsqueda por valor */
    int elegido, esta = 0;
    printf("Ingrese un numero a buscar: ");
    scanf("%d", &elegido);
    for (int i = 0; i < 10; i++)
        if (lista[i] == elegido) { esta = 1; break; }

    if (esta) printf("El numero %d esta en el arreglo.\n", elegido);
    else printf("El numero %d no esta en el arreglo.\n", elegido);
}
```

```
return 0;
}
```

07. Arreglos — Carga, Identificación de Pares y Búsqueda Secuencial

Archivo: clase_5_5_actividad2.c | Carga 8 elementos, muestra cuáles son pares y busca un valor contando sus ocurrencias.

```
#include <stdio.h>

int main() {
    int lista[8];
    int enc = 0;

    /* Cargar el arreglo */
    for (int i = 0; i < 8; i++) {
        printf("Ingresar el elemento %d: ", i);
        scanf("%d", &lista[i]);
    }

    /* Mostrar elementos e indicar pares */
    for (int i = 0; i < 8; i++) {
        printf("%d", lista[i]);
        if (lista[i] % 2 == 0) printf(" <- Es par");
        printf("\n");
    }

    /* Buscar elemento (busqueda lineal) */
    int b;
    printf("\nIngrese el elemento a buscar: ");
    scanf("%d", &b);

    for (int i = 0; i < 8; i++)
        if (lista[i] == b) enc++;

    if (enc == 0) printf("Elemento no encontrado.\n");
    else printf("Elemento encontrado %d veces.\n", enc);

    return 0;
}
```

08. Arreglos — Números Aleatorios con Cuadrado y Cubo

Archivo: actividad2.c | Genera 10 números aleatorios del 1 al 10 y muestra su cuadrado y cubo.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
```

```
int main() {  
    int vector[10];  
    srand(time(NULL));  
  
    for (int i = 0; i < 10; i++)  
        vector[i] = (rand() % 10) + 1;  
  
    printf("Los numeros aleatorios generados son:\n");  
    for (int i = 0; i < 10; i++)  
        printf("Posicion %d: %d\n", i, vector[i]);  
  
    printf("\n%-10s | %-10s | %-10s\n", "Numero", "Cuadrado", "Cubo");  
    printf("-----\n");  
    for (int i = 0; i < 10; i++) {  
        double cuadrado = pow((double)vector[i], 2);  
        double cubo = pow((double)vector[i], 3);  
        printf("%-10d | %-10.0f | %-10.0f\n", vector[i], cuadrado, cubo);  
    }  
  
    return 0;  
}
```

09. Arreglos — Números Primos, Recorrido Inverso y Conteo de Ocurrencias (con Funciones)

Archivo: actividad_28_4.c | Usa funciones separadas para cargar, mostrar en orden inverso, contar primos y contar ocurrencias de un valor.

```
#include <stdio.h>
#include <stdbool.h>
#define TAMANIO 10

void cargarArreglo(int arr[], int n);
void mostrarArreglo(int arr[], int n);
void mostrarInverso(int arr[], int n);
bool esPrimo(int num);
int contarPrimos(int arr[], int n);
int contarOcurrencias(int arr[], int n, int valor);

int main() {
    int miArreglo[TAMANIO];
    int buscado;

    cargarArreglo(miArreglo, TAMANIO);

    printf("\nElementos del arreglo:\n");
    mostrarArreglo(miArreglo, TAMANIO);

    printf("Cantidad de numeros primos: %d\n", contarPrimos(miArreglo, TAMANIO));

    printf("\nArreglo en sentido inverso:\n");
    mostrarInverso(miArreglo, TAMANIO);

    printf("\nIngrese un entero para buscar sus ocurrencias: ");
    scanf("%d", &buscado);
    printf("El numero %d aparece %d veces.\n",
           buscado, contarOcurrencias(miArreglo, TAMANIO, buscado));

    return 0;
}

void cargarArreglo(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("Ingrese el elemento [%d]: ", i);
        scanf("%d", &arr[i]);
    }
}

void mostrarArreglo(int arr[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
```

```

    printf("\n");
}

bool esPrimo(int num) {
    if (num <= 1) return false;
    for (int i = 2; i * i <= num; i++)
        if (num % i == 0) return false;
    return true;
}

int contarPrimos(int arr[], int n) {
    int contador = 0;
    for (int i = 0; i < n; i++)
        if (esPrimo(arr[i])) contador++;
    return contador;
}

void mostrarInverso(int arr[], int n) {
    for (int i = n - 1; i >= 0; i--) printf("%d ", arr[i]);
    printf("\n");
}

int contarOcurrencias(int arr[], int n, int valor) {
    int contador = 0;
    for (int i = 0; i < n; i++)
        if (arr[i] == valor) contador++;
    return contador;
}

```

10. Funciones con Arreglos — Mostrar Elementos y Contar Pares

Archivo: funcion.c | Carga un vector de 5 enteros, lo muestra con una función y cuenta los pares con otra.

```

#include <stdio.h>
#define MAX 5

void mostrar(int a[]);
int pares(int a[]);

int main() {
    int vec[MAX];

    for (int i = 0; i < MAX; i++) {
        printf("Ingrese un numero entero: ");
        scanf("%d", &vec[i]);
    }

    mostrar(vec);
}

```



```

    int p = pares(vec);
    printf("La cantidad de elementos pares es: %d\n", p);

    return 0;
}

void mostrar(int a[]) {
    printf("Elementos del vector: ");
    for (int i = 0; i < MAX; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int pares(int a[]) {
    int cant = 0;
    for (int i = 0; i < MAX; i++)
        if (a[i] % 2 == 0) cant++;
    return cant;
}

```

11. Procedimientos con Arreglos — Mostrar Elementos

Archivo: procedimientos_con_arreglos.c | Carga un vector y lo muestra usando un procedimiento (void).

```

#include <stdio.h>
#define MAX 5

void mostrar(int a[]);

int main() {
    int vec[MAX];

    for (int i = 0; i < MAX; i++) {
        printf("Ingrese un numero entero: ");
        scanf("%d", &vec[i]);
    }

    mostrar(vec);
    return 0;
}

void mostrar(int a[]) {
    for (int i = 0; i < MAX; i++)
        printf("Elemento %d: %d\n", i + 1, a[i]);
}

```

12. Cadenas de Caracteres — Declaración y Recorrido

Archivo: cadenas.c | Declara una cadena carácter a carácter y la recorre con un while.

```
#include <stdio.h>

int main() {
    char cadena[5] = {'h', 'o', 'l', 'a', '\0'};

    printf("La cadena es:\n");
    int i = 0;
    while (i < 5) {
        printf("Caracter %d: %c\n", i, cadena[i]);
        i++;
    }

    return 0;
}
```

13. Cadenas de Caracteres — Carga e Impresión

Archivo: cargar_cadenas.c | Carga una cadena de 5 caracteres con scanf y la muestra.

```
#include <stdio.h>

int main() {
    char cadena[5];

    for (int i = 0; i < 5; i++) {
        printf("Ingrese letra %d: ", i + 1);
        scanf(" %c", &cadena[i]);
    }

    printf("Array: ");
    for (int i = 0; i < 5; i++)
        printf(" %c ", cadena[i]);
    printf("\n");

    return 0;
}
```

14. Búsqueda Binaria en Arreglo Ordenado con Eliminación del Elemento

Archivo: busqueda_binaria.c | Aplica búsqueda binaria sobre un arreglo ordenado y, si lo encuentra, elimina el elemento desplazando el resto.

```
#include <stdio.h>

int main() {
    int lista[9] = {0, 4, 5, 7, 32, 40, 77, 100, 123};
    int n = 9;

    for (int i = 0; i < n; i++)
        printf("Digito %d: %d\n", i, lista[i]);

    /* Búsqueda Binaria */
    int num;
    printf("Ingresar el numero a buscar: ");
    scanf("%d", &num);

    int inicio = 0, fin = n - 1, medio = 0;

    while (inicio <= fin) {
        medio = (inicio + fin) / 2;
        if (num == lista[medio]) break;
        if (num > lista[medio]) inicio = medio + 1;
        else fin = medio - 1;
    }

    if (num == lista[medio]) {
        printf("%d encontrado en la posicion %d\n", num, medio);

        /* Eliminar el elemento desplazando */
        for (int j = medio; j < n - 1; j++)
            lista[j] = lista[j + 1];
        n--;

        printf("Elemento eliminado. Array resultante:\n");
        for (int i = 0; i < n; i++)
            printf("%d ", lista[i]);
        printf("\n");
    } else {
        printf("%d no esta en el arreglo.\n", num);
    }

    return 0;
}
```

15. Búsqueda Binaria — Práctica sobre Arreglo Ordenado Ascendente

Archivo: busqueda_binaria_practica2.c | Implementa búsqueda binaria clásica con variable 'encontrado'.

```
#include <stdio.h>

int main() {
    int lista[8] = {1, 2, 3, 3, 5, 6, 77, 88};
    int b;
    int inicio = 0;
    int fin = 7;
    int encontrado = -1;

    printf("Ingrese elemento a buscar con Búsqueda Binaria: ");
    scanf("%d", &b);

    while (inicio <= fin) {
        int centro = (inicio + fin) / 2;
        if (lista[centro] == b) {
            encontrado = centro;
            break;
        } else if (lista[centro] > b) {
            fin = centro - 1;
        } else {
            inicio = centro + 1;
        }
    }

    if (encontrado != -1)
        printf("Elemento %d encontrado en la posicion %d.\n", b, encontrado);
    else
        printf("Elemento no encontrado.\n");

    return 0;
}
```

16. Búsqueda Binaria con Función — Sumatoria, Promedio y Búsqueda en Arreglo

Archivo: repaso_ordenamiento.c | Calcula sumatoria y promedio, permite buscar por posición y por valor usando una función de búsqueda binaria.

```
#include <stdio.h>

int buscar(int lista[], int num);

int main() {
    int a[10] = {100, 3, 30, 4, 5, 997, 7, 987, 9, 10};
```

```

int acum = 0, pos, bus, enc = 0;
float prom;

/* Mostrar arreglo */
for (int i = 0; i < 10; i++) printf("%d - ", a[i]);
printf("\n");

/* Sumatoria y promedio */
for (int i = 0; i < 10; i++) acum += a[i];
printf("\nTotal sumatoria: %d\n", acum);

prom = (float)acum / 10;
printf("Promedio: %.2f\n", prom);

/* Búsqueda por posición */
printf("\nIngrese posición a buscar (0-9): ");
scanf("%d", &pos);
if (pos < 0 || pos > 9) printf("Posición fuera de rango.\n");
else printf("Valor %d en la posición %d\n", a[pos], pos);

/* Búsqueda por valor */
printf("\nIngrese valor a buscar: ");
scanf("%d", &bus);

int z = buscar(a, bus);
if (z == -1) {
    printf("Elemento no encontrado.\n");
} else {
    while (z < 10 && a[z] == bus) { enc++; z++; }
    printf("Elemento encontrado %d veces.\n", enc);
}

return 0;
}

/* Búsqueda Binaria - devuelve índice o -1 si no existe */
int buscar(int lista[], int num) {
    int inicio = 0, fin = 9, medio = 0;

    while (inicio <= fin) {
        medio = (inicio + fin) / 2;
        if (num == lista[medio]) return medio;
        if (num > lista[medio]) inicio = medio + 1;
        else fin = medio - 1;
    }
    return -1;
}

```

17. Matrices 3x3 — Carga y Visualización

Archivo: matrices.c | Carga una matriz 3x3 con scanf y la muestra en pantalla.

```
#include <stdio.h>

int main() {
    int matriz[3][3];

    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++) {
            printf("Ingrese numero [%d][%d]: ", i, j);
            scanf("%d", &matriz[i][j]);
        }

    printf("\nMatriz:\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++)
            printf("%4d ", matriz[i][j]);
        printf("\n");
    }

    return 0;
}
```

18. Ordenamiento de Matriz 3x3 con Bubble Sort

Archivo: ordenar_matriz.c | Convierte la matriz en un array 1D, la ordena con burbuja y la vuelve a cargar en la matriz.

```
#include <stdio.h>

int main() {
    int matriz[3][3];
    int array[9];
    int k = 0, n = 9, aux;

    /* Ingresar matriz */
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++) {
            printf("Ingrese numero [%d][%d]: ", i, j);
            scanf("%d", &matriz[i][j]);
        }

    /* Convertir a array 1D */
    k = 0;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            array[k++] = matriz[i][j];
}
```

```

/* Ordenamiento Burbuja */
for (int i = 0; i < n - 1; i++)
    for (int j = 0; j < n - 1 - i; j++)
        if (array[j] > array[j + 1]) {
            aux = array[j];
            array[j] = array[j + 1];
            array[j + 1] = aux;
        }

/* Volver a matriz */
k = 0;
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++)
        matriz[i][j] = array[k++];

/* Mostrar */
printf("\nMatriz ordenada:\n");
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++)
        printf("%4d ", matriz[i][j]);
    printf("\n");
}

return 0;
}

```

19. Ordenamiento y Multiplicación de dos Matrices 3×3

Archivo: ordenar_multiplicacion_matrices.c | Ordena dos matrices 3×3 y luego las multiplica.

```

#include <stdio.h>

void matrizAArray(int m[3][3], int a[9]);
void arrayAMatriz(int a[9], int m[3][3]);
void ordenarBurbuja(int a[], int n);

int main() {
    int A[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
    int B[3][3] = {{9,8,7},{6,5,4},{3,2,1}};
    int a1[9], a2[9];

    /* Ordenar A y B */
    matrizAArray(A, a1);
    ordenarBurbuja(a1, 9);
    arrayAMatriz(a1, A);

    matrizAArray(B, a2);
}

```

```

ordenarBurbuja(a2, 9);
arrayAMatriz(a2, B);

/* Mostrar matrices ordenadas */
printf("Matriz A ordenada:\n");
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) printf("%4d", A[i][j]);
    printf("\n");
}
printf("Matriz B ordenada:\n");
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) printf("%4d", B[i][j]);
    printf("\n");
}

/* Multiplicacion A x B */
int C[3][3];
for (int x = 0; x < 3; x++)
    for (int y = 0; y < 3; y++) {
        C[x][y] = 0;
        for (int k = 0; k < 3; k++)
            C[x][y] += A[x][k] * B[k][y];
    }

printf("Resultado A x B:\n");
for (int x = 0; x < 3; x++) {
    for (int y = 0; y < 3; y++) printf("%6d", C[x][y]);
    printf("\n");
}

return 0;
}

void matrizAArray(int m[3][3], int a[9]) {
    int k = 0;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            a[k++] = m[i][j];
}

void arrayAMatriz(int a[9], int m[3][3]) {
    int k = 0;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            m[i][j] = a[k++];
}

```



```
void ordenarBurbuja(int a[], int n) {  
    int aux;  
    for (int i = 0; i < n - 1; i++)  
        for (int j = 0; j < n - 1 - i; j++)  
            if (a[j] > a[j + 1]) {  
                aux = a[j]; a[j] = a[j+1]; a[j+1] = aux;  
            }  
}
```

20. Matrices 3x3 — Carga sin Repetidos, Máximo, Mínimo y Diagonal

Principal

Archivo: ejercicio4_matrices.c | Carga una matriz sin valores repetidos usando recursión, busca máximo, mínimo y extrae la diagonal principal.

```
#include <stdio.h>

int ya_existe(int usados[], int cantidad, int num) {
    for (int i = 0; i < cantidad; i++)
        if (usados[i] == num) return 1;
    return 0;
}

void pedir_numero(int matriz[3][3], int i, int j, int usados[], int cantidad) {
    int num;
    printf("Ingrese numero para [%d][%d]: ", i, j);
    scanf("%d", &num);
    if (ya_existe(usados, cantidad, num)) {
        printf("El numero %d ya existe. Ingrese otro.\n", num);
        pedir_numero(matriz, i, j, usados, cantidad);
    } else {
        matriz[i][j] = num;
        usados[cantidad] = num;
    }
}

int main() {
    int matriz[3][3];
    int usados[9];
    int cantidad = 0;
    int min = 9999, max = -9999;
    int fila_min = 0, col_min = 0, fila_max = 0, col_max = 0;

    /* Carga sin repetidos */
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++) {
            pedir_numero(matriz, i, j, usados, cantidad);
            cantidad++;
        }

    /* Mostrar */
    printf("\nMatriz ingresada:\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) printf("%4d ", matriz[i][j]);
        printf("\n");
    }
}
```

```

/* Buscar min y max */
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++) {
        if (matriz[i][j] < min) { min = matriz[i][j]; fila_min = i; col_min = j; }
        if (matriz[i][j] > max) { max = matriz[i][j]; fila_max = i; col_max = j; }
    }

printf("Valor MAYOR: %d en [%d][%d]\n", max, fila_max, col_max);
printf("Valor MENOR: %d en [%d][%d]\n", min, fila_min, col_min);

/* Diagonal principal */
printf("Diagonal principal: ");
for (int i = 0; i < 3; i++) printf("%d ", matriz[i][i]);
printf("\n");

return 0;
}

```

21. Tabla de Máximo Común Divisor (MCD) con los Divisores de 50

Archivo: ejercicio5_guia2.c | Genera una tabla del MCD usando los divisores de 50 como filas y columnas.

```

#include <stdio.h>

int calcular_mcd(int a, int b) {
    int menor = (a < b) ? a : b;
    for (int i = menor; i >= 1; i--)
        if (a % i == 0 && b % i == 0) return i;
    return 1;
}

int main() {
    int D50[6] = {1, 2, 5, 10, 25, 50};

    /* Encabezado */
    printf("MCD\t");
    for (int j = 0; j < 6; j++) printf(" %3d\t", D50[j]);
    printf("\n");
    printf("-----");
    for (int j = 0; j < 6; j++) printf("-----");
    printf("\n");

    /* Tabla */
    for (int i = 0; i < 6; i++) {
        printf("%3d\t", D50[i]);
        for (int j = 0; j < 6; j++)
            printf(" %3d\t", calcular_mcd(D50[i], D50[j]));
        printf("\n");
    }
}

```

```
}
```

```
return 0;
```

```
}
```

22. Arreglos A y B — Suma en C, Ordenamiento, Unión sin Repetidos y Búsqueda Binaria

Archivo: actividad_12_5.c | Carga B, suma A+B en C, ordena ambos descendente, une en D sin repetidos y hace búsqueda binaria en D.

```
#include <stdio.h>

void ordenarDescendente(int lista[], int n);
int buscarBinarioDesc(int lista[], int n, int num);

int main() {
    int A[10] = {10, 3, 30, 4, 5, 2, 7, 1, 9, 70};
    int B[10], C[10], D[20];
    int sizeD = 0, n = 10;

    /* Cargar B */
    for (int i = 0; i < n; i++) {
        printf("Ingrese numero B[%d]: ", i + 1);
        scanf("%d", &B[i]);
    }

    /* Suma A + B en C */
    printf("\nContenido de C (A + B):\n");
    for (int i = 0; i < n; i++) {
        C[i] = A[i] + B[i];
        printf("C[%d] = %d\n", i, C[i]);
    }

    /* Ordenar A y B descendente */
    ordenarDescendente(A, n);
    ordenarDescendente(B, n);

    printf("\nA ordenada (desc): ");
    for (int i = 0; i < n; i++) printf("%d ", A[i]);
    printf("\nB ordenada (desc): ");
    for (int i = 0; i < n; i++) printf("%d ", B[i]);
    printf("\n");

    /* Union sin repetidos en D */
    for (int i = 0; i < n; i++) {
        int rep = 0;
        for (int k = 0; k < sizeD; k++) if (A[i] == D[k]) { rep = 1; break; }
        if (!rep) D[sizeD++] = A[i];
    }
    for (int i = 0; i < n; i++) {
        int rep = 0;
        for (int k = 0; k < sizeD; k++) if (B[i] == D[k]) { rep = 1; break; }
```

```

        if (!rep) D[sizeD++] = B[i];
    }
    ordenarDescendente(D, sizeD);

    printf("\nD (union sin repetidos, descendente):\n");
    for (int i = 0; i < sizeD; i++) printf("%d ", D[i]);
    printf("\n");

    /* Busqueda Binaria en D */
    int b;
    printf("\nIngrese elemento a buscar en D: ");
    scanf("%d", &b);
    int pos = buscarBinarioDesc(D, sizeD, b);
    if (pos != -1) printf("Elemento %d encontrado en posicion %d de D.\n", b, pos);
    else printf("Elemento no encontrado.\n");

    return 0;
}

void ordenarDescendente(int lista[], int n) {
    int aux;
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)
            if (lista[j] < lista[j + 1]) {
                aux = lista[j]; lista[j] = lista[j+1]; lista[j+1] = aux;
            }
}

int buscarBinarioDesc(int lista[], int n, int num) {
    int inicio = 0, fin = n - 1;
    while (inicio <= fin) {
        int centro = (inicio + fin) / 2;
        if (lista[centro] == num) return centro;
        if (lista[centro] < num) fin = centro - 1;
        else inicio = centro + 1;
    }
    return -1;
}

```

23. Arreglos — Unión Ordenada Descendente y Búsqueda Binaria (Versión Corregida)

Archivo: ejercicio1_corregido_12_5.c | Versión corregida del ejercicio anterior con funciones bien separadas para ordenar y buscar.

```

#include <stdio.h>

void ordenarBurbujaDescendente(int lista[], int n);

```

```

int buscarBinarioDescendente(int lista[], int n, int num);

int main() {
    int A[10] = {10, 3, 30, 4, 5, 2, 7, 1, 9, 70};
    int B[10], C[10], D[20];
    int sizeD = 0, n = 10;

    for (int i = 0; i < n; i++) {
        printf("Ingrese numero para B[%d]: ", i + 1);
        scanf("%d", &B[i]);
    }

    printf("\nC (A + B): ");
    for (int i = 0; i < n; i++) { C[i] = A[i] + B[i]; printf("%d ", C[i]); }
    printf("\n");

    ordenarBurbujaDescendente(A, n);
    ordenarBurbujaDescendente(B, n);

    printf("A ordenada (desc): "); for (int i=0;i<n;i++) printf("%d ",A[i]);
    printf("\n");
    printf("B ordenada (desc): "); for (int i=0;i<n;i++) printf("%d ",B[i]);
    printf("\n");

    /* Union sin repetidos */
    for (int i = 0; i < n; i++) {
        int rep = 0;
        for (int k = 0; k < sizeD; k++) if (A[i]==D[k]) { rep=1; break; }
        if (!rep) D[sizeD++] = A[i];
    }
    for (int i = 0; i < n; i++) {
        int rep = 0;
        for (int k = 0; k < sizeD; k++) if (B[i]==D[k]) { rep=1; break; }
        if (!rep) D[sizeD++] = B[i];
    }
    ordenarBurbujaDescendente(D, sizeD);

    printf("\nD (union ordenada desc sin repetidos):\n");
    for (int i = 0; i < sizeD; i++) printf("%d ", D[i]);
    printf("\n");

    int b;
    printf("\nIngrese elemento a buscar en D: ");
    scanf("%d", &b);
    int pos = buscarBinarioDescendente(D, sizeD, b);
    if (pos != -1) printf("Elemento %d encontrado en indice %d de D.\n", b, pos);
    else printf("Elemento no encontrado en D.\n");
}

```

```
    return 0;
}

void ordenarBurbujaDescendente(int lista[], int n) {
    int aux;
    for (int i = 0; i < n-1; i++)
        for (int j = 0; j < n-1-i; j++)
            if (lista[j] < lista[j+1]) {
                aux=lista[j]; lista[j]=lista[j+1]; lista[j+1]=aux;
            }
}

int buscarBinarioDescendente(int lista[], int n, int num) {
    int inicio = 0, fin = n - 1;
    while (inicio <= fin) {
        int centro = (inicio + fin) / 2;
        if (lista[centro] == num) return centro;
        if (lista[centro] < num) fin = centro - 1;
        else inicio = centro + 1;
    }
    return -1;
}
```


24. Structs — Sistema Básico de Libros y Autores con Menú

Archivo: funciones_y_procedimientos.c | Menú de una sola opción para cargar autores, cargar libros o listar libros. El struct Libro contiene un struct Autor anidado.

```
#include <stdio.h>
#define CANT_LIBROS 3
#define CANT_AUTORES 2
#define MAX_TEXTO 100

struct Autor {
    char nombre[MAX_TEXTO];
    char nacionalidad[MAX_TEXTO];
    int cantLibrosPublicados;
};

struct Libro {
    char titulo[MAX_TEXTO];
    char editorial[MAX_TEXTO];
    char genero[MAX_TEXTO];
    int isbn;
    int anioPublicacion;
    struct Autor autor;
};

void cargarAutor(struct Autor autores[], int i);
void cargarLibro(struct Libro libros[], struct Autor autores[], int i);
void listarLibros(struct Libro libros[], int cantidad);

int main() {
    struct Autor autores[CANT_AUTORES];
    struct Libro libros[CANT_LIBROS];
    int opcion;

    printf("1. Cargar autor 2. Cargar libros 3. Listar libros\n");
    printf("Ingrese opcion: ");
    scanf(" %d", &opcion);
    getchar();

    switch (opcion) {
        case 1:
            for (int i = 0; i < CANT_AUTORES; i++) cargarAutor(autores, i);
            break;
        case 2:
            for (int i = 0; i < CANT_LIBROS; i++) cargarLibro(libros, autores, i);
            break;
        case 3:
            listarLibros(libros, CANT_LIBROS);
    }
}
```

```

        break;
    default:
        printf("Opcion invalida.\n");
    }
    return 0;
}

void cargarAutor(struct Autor autores[], int i) {
    printf("Nombre del autor %d: ", i + 1); fgets(autores[i].nombre, MAX_TEXT0,
stdn);
    printf("Nacionalidad del autor %d: ", i + 1); fgets(autores[i].nacionalidad,
MAX_TEXT0, stdn);
    printf("Libros publicados autor %d: ", i + 1); scanf(" %d",
&autores[i].cantLibrosPublicados);
    getchar();
}

void cargarLibro(struct Libro libros[], struct Autor autores[], int i) {
    printf("Titulo libro %d: ", i + 1); fgets(libros[i].titulo, MAX_TEXT0, stdn);
    printf("Editorial libro %d: ", i + 1); fgets(libros[i].editorial, MAX_TEXT0,
stdn);
    printf("Genero libro %d: ", i + 1); fgets(libros[i].genero, MAX_TEXT0, stdn);
    printf("ISBN libro %d: ", i + 1); scanf("%d", &libros[i].isbn);
    printf("Anio libro %d: ", i + 1); scanf(" %d", &libros[i].anioPublicacion);
    getchar();
    int idAutor;
    printf("ID del autor (1-%d): ", CANT_AUTORES); scanf(" %d", &idAutor); getchar();
    libros[i].autor = autores[idAutor - 1];
}

void listarLibros(struct Libro libros[], int cantidad) {
    printf("=== LISTADO DE LIBROS ===\n");
    for (int i = 0; i < cantidad; i++) {
        printf("Titulo : %s", libros[i].titulo);
        printf("Editorial : %s", libros[i].editorial);
        printf("Genero : %s", libros[i].genero);
        printf("ISBN : %d\n", libros[i].isbn);
        printf("Anio : %d\n", libros[i].anioPublicacion);
        printf("Autor : %s", libros[i].autor.nombre);
        printf("Nac. autor : %s", libros[i].autor.nacionalidad);
        printf("Libros pub.: %d\n", libros[i].autor.cantLibrosPublicados);
        printf("-----\n");
    }
}

```

25. Structs — Gestión de Libros y Autores con Funciones Separadas

Archivo: funciones_y_procedimientos_libro.c | Igual al anterior pero sin menú: carga autores, luego libros y finalmente lista todo.

```
#include <stdio.h>

#define CANT_LIBROS 3
#define CANT_AUTORES 2
#define MAX_TEXTO 100

struct Autor { char nombre[MAX_TEXTO]; char nacionalidad[MAX_TEXTO]; int
cantLibrosPublicados; };
struct Libro { char titulo[MAX_TEXTO]; char editorial[MAX_TEXTO]; char
genero[MAX_TEXTO];
                int isbn; int anioPublicacion; struct Autor autor; };

void cargarAutor(struct Autor autores[], int i);
void cargarLibro(struct Libro libros[], struct Autor autores[], int i);
void listarLibros(struct Libro libros[], int cantidad);

int main() {
    struct Autor autores[CANT_AUTORES];
    struct Libro libros[CANT_LIBROS];

    printf("=== CARGA DE AUTORES ===\n");
    for (int i = 0; i < CANT_AUTORES; i++) cargarAutor(autores, i);

    printf("=== CARGA DE LIBROS ===\n");
    for (int i = 0; i < CANT_LIBROS; i++) cargarLibro(libros, autores, i);

    listarLibros(libros, CANT_LIBROS);
    return 0;
}

void cargarAutor(struct Autor autores[], int i) {
    printf("Nombre autor %d : ", i+1); fgets(autores[i].nombre, MAX_TEXTO, stdin);
    printf("Nacionalidad autor %d: ", i+1); fgets(autores[i].nacionalidad, MAX_TEXTO,
stdin);
    printf("Libros publicados %d : ", i+1); scanf(" %d",
&autores[i].cantLibrosPublicados); getchar();
}

void cargarLibro(struct Libro libros[], struct Autor autores[], int i) {
    printf("Titulo libro %d : ", i+1); fgets(libros[i].titulo, MAX_TEXTO, stdin);
    printf("Editorial libro %d: ", i+1); fgets(libros[i].editorial, MAX_TEXTO,
stdin);
    printf("Genero libro %d : ", i+1); fgets(libros[i].genero, MAX_TEXTO, stdin);
    printf("ISBN libro %d : ", i+1); scanf("%d", &libros[i].isbn);
    printf("Anio libro %d : ", i+1); scanf(" %d", &libros[i].anioPublicacion);
    getchar();
    int id; printf("ID autor (1-%d) : ", CANT_AUTORES); scanf(" %d", &id); getchar();
```

```
    libros[i].autor = autores[id - 1];
}

void listarLibros(struct Libro libros[], int cantidad) {
    printf("\n=== LISTADO DE LIBROS ===\n");
    for (int i = 0; i < cantidad; i++) {
        printf("Titulo: %s", libros[i].titulo);
        printf("Editorial: %s", libros[i].editorial);
        printf("ISBN: %d | Anio: %d\n", libros[i].isbn, libros[i].anioPublicacion);
        printf("Autor: %s | Nac.: %s | Publicados: %d\n",
            libros[i].autor.nombre, libros[i].autor.nacionalidad,
            libros[i].autor.cantLibrosPublicados);
        printf("-----\n");
    }
}
```

26. Structs — Sistema Completo de Libros y Autores con CRUD y Menú

Archivo: funciones_y_procedimientos_profe.c | Menú do-while completo con hasta 1000 libros y autores. Permite cargar, listar y buscar autores por ID.

```
#include <stdio.h>
#define CANT_LIBROS 1000
#define CANT_AUTORES 1000
#define MAX_TEXTO 100

struct Autor { char nombre[MAX_TEXTO]; char nacionalidad[MAX_TEXTO]; int
cantLibrosPublicados; };
struct Libro { char titulo[MAX_TEXTO]; char editorial[MAX_TEXTO]; char
genero[MAX_TEXTO];
                int isbn; int anioPublicacion; struct Autor autor; };

void cargarAutor(struct Autor autores[], int i);
void cargarLibro(struct Libro libros[], struct Autor autores[], int i);
void listarLibros(struct Libro libros[], int cantidad);
void listarAutores(struct Autor autores[], int cantidad);
void mostrarAutor(struct Autor autor);

int main() {
    struct Autor autores[CANT_AUTORES];
    struct Libro libros[CANT_LIBROS];
    int cantAutores = 0, cantLibros = 0, opcion = 0;

    do {
        printf("\n=== MENU DE LIBROS ===\n");
        printf("1. Cargar autor\n2. Cargar libro\n3. Listar libros\n");
        printf("4. Listar autores\n5. Buscar autor por ID\n9. Salir\n");
        printf("Seleccione: ");
        scanf(" %d", &opcion);
        getchar();

        switch (opcion) {
            case 1: cargarAutor(autores, cantAutores++); break;
            case 2: cargarLibro(libros, autores, cantLibros++); break;
            case 3: listarLibros(libros, cantLibros); break;
            case 4: listarAutores(autores, cantAutores); break;
            case 5: {
                int id;
                printf("Ingrese ID del autor (0 a %d): ", cantAutores - 1);
                scanf(" %d", &id);
                if (id >= 0 && id < cantAutores) mostrarAutor(autores[id]);
                else printf("ID fuera de rango.\n");
                break;
            }
            case 9: printf("Saliendo...\n"); break;
        }
    } while (opcion != 9);
}
```

```

        default: printf("Opcion invalida.\n");
    }
} while (opcion != 9);
return 0;
}

void cargarAutor(struct Autor autores[], int i) {
    printf("Nombre autor %d : ", i+1); fgets(autores[i].nombre, MAX_TEXTO, stdin);
    printf("Nacionalidad autor %d: ", i+1); fgets(autores[i].nacionalidad, MAX_TEXTO,
stdin);
    printf("Libros publicados %d : ", i+1); scanf(" %d",
&autores[i].cantLibrosPublicados); getchar();
}

void cargarLibro(struct Libro libros[], struct Autor autores[], int i) {
    printf("Titulo libro %d : ", i+1); fgets(libros[i].titulo, MAX_TEXTO, stdin);
    printf("Editorial libro %d: ", i+1); fgets(libros[i].editorial, MAX_TEXTO,
stdin);
    printf("Genero libro %d : ", i+1); fgets(libros[i].genero, MAX_TEXTO, stdin);
    printf("ISBN libro %d : ", i+1); scanf("%d", &libros[i].isbn);
    printf("Anio libro %d : ", i+1); scanf(" %d", &libros[i].anioPublicacion);
getchar();
    int id; printf("ID autor: "); scanf(" %d", &id); getchar();
    libros[i].autor = autores[id];
}

void listarLibros(struct Libro libros[], int cantidad) {
    printf("=== LISTADO DE LIBROS ===\n");
    for (int i = 0; i < cantidad; i++) {
        printf("[%d] %s | ISBN: %d | Autor: %s\n",
            i, libros[i].titulo, libros[i].isbn, libros[i].autor.nombre);
    }
}

void listarAutores(struct Autor autores[], int cantidad) {
    printf("=== LISTADO DE AUTORES ===\n");
    for (int i = 0; i < cantidad; i++) {
        printf("[%d] %s | Nac.: %s | Publicados: %d\n",
            i, autores[i].nombre, autores[i].nacionalidad,
autores[i].cantLibrosPublicados);
    }
}

void mostrarAutor(struct Autor autor) {
    printf("Nombre : %s", autor.nombre);
    printf("Nac. : %s", autor.nacionalidad);
    printf("Publicados : %d\n", autor.cantLibrosPublicados);
}

```

27. Structs — Carga de Autores y Libros con Struct Anidado

Archivo: struct_libros.c | Versión directa sin menú: carga autores y luego libros con el struct Autor anidado directamente dentro de Libro.

```
#include <stdio.h>
#define CANT_LIBROS 3
#define CANT_AUTORES 2
#define MAX_TEXTO 100

struct Autor { char nombre[MAX_TEXTO]; char nacionalidad[MAX_TEXTO]; int
cantLibrosPublicados; };
struct Libro { char titulo[MAX_TEXTO]; struct Autor autor;
char editorial[MAX_TEXTO]; char genero[MAX_TEXTO];
int isbn; int anioPublicacion; };

int main() {
    struct Autor autores[CANT_AUTORES];

    printf("=== CARGA DE AUTORES ===\n");
    for (int i = 0; i < CANT_AUTORES; i++) {
        printf("Nombre autor %d : ", i+1); fgets(autores[i].nombre, MAX_TEXTO,
stdin);
        printf("Nacionalidad autor %d: ", i+1); fgets(autores[i].nacionalidad,
MAX_TEXTO, stdin);
        printf("Libros publicados %d : ", i+1);
        scanf("%d", &autores[i].cantLibrosPublicados); getchar();
    }

    struct Libro libros[CANT_LIBROS];

    printf("=== CARGA DE LIBROS ===\n");
    for (int i = 0; i < CANT_LIBROS; i++) {
        printf("Titulo libro %d : ", i+1); fgets(libros[i].titulo, MAX_TEXTO, stdin);
        printf("Nombre del autor libro %d : ", i+1); fgets(libros[i].autor.nombre,
MAX_TEXTO, stdin);
        printf("Nac. del autor libro %d : ", i+1);
        fgets(libros[i].autor.nacionalidad, MAX_TEXTO, stdin);
        printf("Libros pub. por autor %d : ", i+1);
        scanf("%d", &libros[i].autor.cantLibrosPublicados); getchar();
        printf("Editorial libro %d : ", i+1); fgets(libros[i].editorial, MAX_TEXTO,
stdin);
        printf("Genero libro %d : ", i+1); fgets(libros[i].genero, MAX_TEXTO, stdin);
        printf("ISBN libro %d : ", i+1); scanf("%d", &libros[i].isbn); getchar();
        printf("Anio publicacion libro %d : ", i+1); scanf("%d",
&libros[i].anioPublicacion); getchar();
    }

    return 0;
}
```


28. Structs — Registro de Clientes con Ordenamiento Burbuja

Archivo: ejercicio5_guia.c | Carga 5 clientes (nro, tipo E/P, nombre, celular), ordena por número de cliente y los muestra en una tabla.

```
#include <stdio.h>
#include <string.h>
#define MAX 5

typedef struct {
    int nro_cliente;
    char tipo; /* 'E' = Empresa, 'P' = Particular */
    char nombre[50];
    char celular[20];
} Cliente;

void cargar(Cliente lista[], int n);
void ordenar(Cliente lista[], int n);
void mostrar(Cliente lista[], int n);

int main() {
    Cliente clientes[MAX];
    cargar(clientes, MAX);
    ordenar(clientes, MAX);
    mostrar(clientes, MAX);
    return 0;
}

void cargar(Cliente lista[], int n) {
    for (int i = 0; i < n; i++) {
        printf("\n--- Cliente %d ---\n", i + 1);
        printf("Nro de cliente : "); scanf("%d", &lista[i].nro_cliente);
        do {
            printf("Tipo (E/P) : "); scanf(" %c", &lista[i].tipo);
            if (lista[i].tipo != 'E' && lista[i].tipo != 'P')
                printf("Tipo invalido. Ingrese E o P.\n");
        } while (lista[i].tipo != 'E' && lista[i].tipo != 'P');
        printf("Nombre : "); scanf(" %s", lista[i].nombre);
        printf("Nro Celular : "); scanf(" %s", lista[i].celular);
    }
}

void ordenar(Cliente lista[], int n) {
    Cliente temp;
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)
            if (lista[j].nro_cliente > lista[j + 1].nro_cliente) {
                temp = lista[j];
```

```

        lista[j] = lista[j + 1];
        lista[j + 1] = temp;
    }
}

void mostrar(Cliente lista[], int n) {
    printf("\n%-6s %-10s %-20s %-15s\n", "Nro", "Tipo", "Nombre", "Celular");
    printf("-----\n");
    for (int i = 0; i < n; i++) {
        char *tipo_str = (lista[i].tipo == 'E') ? "Empresa" : "Particular";
        printf("%-6d %-10s %-20s %-15s\n",
            lista[i].nro_cliente, tipo_str, lista[i].nombre, lista[i].celular);
    }
}

```

29. Structs — Clientes con Búsqueda Binaria y Separación por Tipo

Archivo: ejercicio5_practica.c | Agrega búsqueda binaria por nro_cliente y separación de clientes en dos arreglos: empresas y particulares.

```

#include <stdio.h>
#include <string.h>
#define MAX 5

typedef struct {
    int nro_cliente;
    char tipo;
    char nombre[50];
    char celular[20];
} Cliente;

void cargar(Cliente lista[], int n);
void ordenar(Cliente lista[], int n);
void mostrar(Cliente lista[], int n);
int busqueda_binaria(Cliente lista[], int n, int nro_buscado);
void separar_por_tipo(Cliente lista[], int n,
    Cliente empresas[], int *cant_e,
    Cliente particulares[], int *cant_p);

int main() {
    Cliente clientes[MAX], empresas[MAX], particulares[MAX];
    int cant_e = 0, cant_p = 0;

    cargar(clientes, MAX);
    ordenar(clientes, MAX);
    mostrar(clientes, MAX);

    /* Busqueda Binaria */
}

```

```

    int nro;
    printf("\nIngrese Nro de cliente a buscar: ");
    scanf("%d", &nro);
    int res = busqueda_binaria(clientes, MAX, nro);
    if (res == -1) {
        printf("Cliente %d no encontrado.\n", nro);
    } else {
        printf("\nCliente encontrado:\n");
        printf("Nro : %d\n", clientes[res].nro_cliente);
        printf("Tipo : %s\n", (clientes[res].tipo=='E') ? "Empresa" : "Particular");
        printf("Nombre : %s\n", clientes[res].nombre);
        printf("Cel. : %s\n", clientes[res].celular);
    }

    /* Separar por tipo */
    separar_por_tipo(clientes, MAX, empresas, &cant_e, particulares, &cant_p);
    printf("\n=== Clientes EMPRESA (%d) ===\n", cant_e); mostrar(empresas, cant_e);
    printf("\n=== Clientes PARTICULAR (%d) ===\n", cant_p); mostrar(particulares, cant_p);

    return 0;
}

void cargar(Cliente lista[], int n) {
    for (int i = 0; i < n; i++) {
        printf("\n--- Cliente %d ---\n", i+1);
        printf("Nro: "); scanf("%d", &lista[i].nro_cliente);
        do {
            printf("Tipo (E/P): "); scanf(" %c", &lista[i].tipo);
            if (lista[i].tipo!='E' && lista[i].tipo!='P') printf("Invalido.\n");
        } while (lista[i].tipo!='E' && lista[i].tipo!='P');
        printf("Nombre : "); scanf(" %[^\n]", lista[i].nombre);
        printf("Celular: "); scanf(" %[^\n]", lista[i].celular);
    }
}

void ordenar(Cliente lista[], int n) {
    Cliente temp;
    for (int i=0;i<n-1;i++) for (int j=0;j<n-1-i;j++)
        if (lista[j].nro_cliente>lista[j+1].nro_cliente) {
            temp=lista[j]; lista[j]=lista[j+1]; lista[j+1]=temp;
        }
}

void mostrar(Cliente lista[], int n) {
    printf("%-6s %-10s %-20s %-15s\n", "Nro", "Tipo", "Nombre", "Celular");
    printf("-----\n");
    for (int i=0;i<n;i++) {

```

```

        printf("%-6d %-10s %-20s %-15s\n", lista[i].nro_cliente,
               (lista[i].tipo=='E')?"Empresa":"Particular", lista[i].nombre,
               lista[i].celular);
    }
}

int busqueda_binaria(Cliente lista[], int n, int nro) {
    int inicio=0, fin=n-1;
    while (inicio<=fin) {
        int medio=(inicio+fin)/2;
        if (lista[medio].nro_cliente==nro) return medio;
        if (lista[medio].nro_cliente< nro) inicio=medio+1;
        else fin =medio-1;
    }
    return -1;
}

void separar_por_tipo(Cliente lista[], int n,
                     Cliente empresas[], int *cant_e,
                     Cliente particulares[], int *cant_p) {
    *cant_e=0; *cant_p=0;
    for (int i=0;i<n;i++) {
        if (lista[i].tipo=='E') empresas[(*cant_e)++]=lista[i];
        else particulares[(*cant_p)++]=lista[i];
    }
}

```

30. Structs — Cuentas Bancarias con Informe de Deudores y Acreedores

Archivo: struct_cuentas_bancarias.c | Carga cuentas bancarias (nro, nombre, saldo), muestra un listado con total y clasifica a cada cliente como deudor o acreedor.

```

#include <stdio.h>
#define NRO_CUENTAS 3

struct Cuenta {
    int nro_cuenta;
    char nombre[30];
    int saldo;
};

int main() {
    struct Cuenta cuen[NRO_CUENTAS];
    int deudor = 0, acreedor = 0, total = 0;

    /* Carga */
    for (int i = 0; i < NRO_CUENTAS; i++) {
        printf("=== Estructura %d ===\n", i + 1);
    }
}

```

```

    printf("Nro de Cuenta : "); scanf("%d", &cuen[i].nro_cuenta);
    printf("Nombre : "); scanf(" %29[^\n]", cuen[i].nombre);
    printf("Saldo : "); scanf("%d", &cuen[i].saldo);
    printf("\n");
}

/* Listado */
printf("-----\n");
printf(" Listado de Clientes \n");
printf("-----\n");
printf("%-12s %-20s %-10s\n", "Nro Cuenta", "Nombre", "Saldo");
printf("-----\n");
for (int i = 0; i < NRO_CUENTAS; i++) {
    printf("%-12d %-20s %-10d\n", cuen[i].nro_cuenta, cuen[i].nombre,
cuen[i].saldo);
    total += cuen[i].saldo;
}
printf("-----\n");
printf("Total Saldo: %d\n", total);

/* Estado de cuentas */
printf("\n=== Estado de Cuentas ===\n");
for (int i = 0; i < NRO_CUENTAS; i++) {
    if (cuen[i].saldo < 0) {
        printf("Cliente %-20s -> DEUDOR\n", cuen[i].nombre);
        deudor++;
    } else {
        printf("Cliente %-20s -> ACREEDOR\n", cuen[i].nombre);
        acreedor++;
    }
}
printf("\nTotal deudores : %d\n", deudor);
printf("Total acreedores: %d\n", acreedor);

return 0;
}

```